

IP Inventory Management System: Setup and Operations Guide

Version 1.0 | October 2026

1. What the system does

A web application for managing VLANs and the IP addresses inside them.

- An **Admin** creates a VLAN by entering the network address and CIDR (for example 10.10.20.0/24). The system calculates the range, previews it, checks for duplicates and overlaps, and automatically creates every usable IP record.
- A **Normal User** picks an available IP and assigns it to equipment, with room, department, equipment ID and name, CPU S/N, instrument S/N, user name and remarks. Assignments can be edited, set inactive, or released.
- Every change is recorded in an **audit log** showing who did what and when.
- A **dashboard** shows utilization per VLAN, and the IP list can be **exported to CSV** (opens in Excel).

Roles

Capability	Admin	Normal User
Dashboard, IP list, search, filters, CSV export	Yes	Yes
Assign, edit, release, set inactive/reactivate IPs	Yes	Yes
Change own password	Yes	Yes
Create and edit VLANs	Yes	No
Reserve / unreserve IPs	Yes	No
Manage users (add, deactivate, reset password, change role)	Yes	No
View audit log	Yes	No

Key rules built into the system

- Network and broadcast addresses are **not** stored as IP records. Only usable host addresses are created.
- The gateway is reserved automatically and can never be changed. Extra reserved IPs can be listed when creating the VLAN.
- New IPs start as **Available**. Only Available IPs can be assigned.
- The network address must be the first address of its subnet (10.0.11.0/24, not 10.0.11.1/24).
- Allowed CIDR sizes are /16 to /30. VLAN ID must be 1 to 4094 and unique.
- Overlapping or duplicate networks are rejected, including a smaller network inside an existing larger one and the reverse.
- Network, CIDR and gateway cannot be edited after creation, because they define the generated IP records.
- Two people choosing the same IP at the same moment: the second gets a message that the IP is no longer available.
- A deactivated user is locked out immediately. An Admin cannot deactivate or demote their own account.

2. Architecture

```
Browser --HTTP--> Backend service (FastAPI + Uvicorn, port 8000) --> MySQL 8
                    also serves the built React website
```

One program serves both the website and the API on port 8000. It runs as a Windows service (NSSM), so it starts at boot and needs no open windows.

3. Requirements for the dedicated PC

Item	Requirement
Operating system	Windows 10/11 or Windows Server (commands below use Windows cmd)
Python	3.10 or newer (tick Add python.exe to PATH during install)
MySQL Server	8.0.16 or newer (needed for CHECK constraints)
Node.js	18 LTS or newer (only to build the website once; not needed to run it)
NSSM	From nssm.cc (the Windows service wrapper)
Network	Fixed IP address for the PC; TCP port 8000 allowed in the firewall
Hardware	Any modern PC; 4 GB RAM and 10 GB free disk is plenty

4. Package contents

```
IP_Inventory\
├── backend\
│   ├── requirements.txt
│   ├── set_password.py          (set or reset an app user's password)
│   ├── backup.bat              (database backup script)
│   └── app\                    (main, db, security, auth, users, vlans, ips, edit,
audit, dashboard, ipcalc)
├── frontend\
│   ├── package.json, index.html, vite.config.js
│   └── src\                    (main.jsx, App.jsx, styles.css)
├── database\
│   └── setup_database.sql      (complete database, run once)
└── docs\
    └── IP_Inventory_Setup_Guide.md
```

5. Installation on a new PC

Use `D:\IP_Inventory_Mgmt` as the install folder in the examples (any folder works; adjust the paths).

Step 1: Install the software

Install Python, MySQL Server, Node.js. Remember the MySQL root password you choose. Download `nssm.exe` (win64) and place it in `D:\IP_Inventory_Mgmt\tools\`.

Step 2: Copy the project

Copy the `backend`, `frontend`, `database` and `docs` folders into `D:\IP_Inventory_Mgmt\`.

Step 3: Create the database

```
cd D:\IP_Inventory_Mgmt\database
mysql -u root -p < setup_database.sql
```

Check it:

```
mysql -u root -p -e "USE ip_inventory; SHOW TABLES;"
```

Expected tables: audit_logs, ip_addresses, users, vlans. The script also creates the first user `admin` with a placeholder password.

Step 4: Create the Python environment and install packages

```
cd D:\IP_Inventory_Mgmt
python -m venv venv
venv\Scripts\activate
pip install -r backend\requirements.txt
```

Step 5: Test the backend by hand

The application reads its settings from two environment variables:

Variable	Meaning
DATABASE_URL	mysql+pymysql://USER:PASSWORD@localhost:3306/ip_inventory
JWT_SECRET	A long random text (32+ characters) that signs login tokens. Keep it private.

In the URL, special characters in the password must be encoded: @ becomes %40, # becomes %23, / becomes %2F.

Generate a secret (it prints on screen; do not share it):

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Run the test **in one window** (the `set` lines only last for that window):

```
cd D:\IP_Inventory_Mgmt\backend
D:\IP_Inventory_Mgmt\venv\Scripts\activate
set DATABASE_URL=mysql+pymysql://root:YOUR_ENCODED_PASSWORD@localhost:3306/ip_inventory
set JWT_SECRET=YOUR_LONG_RANDOM_SECRET
python -c "from app.db import engine; c=engine.connect(); print('DB OK, users:', c.exec_driver_sql('select count(*) from users').scalar())"
python set_password.py admin YourAdminPassword
python -m uvicorn app.main:app
```

You need `DB OK, users: 1`, then `Password updated.`, then `Application startup complete.` Open `http://localhost:8000/api/health` and expect `{"status": "ok"}`. Stop the test with `Ctrl+C`.

Step 6: Build the website

```
cd D:\IP_Inventory_Mgmt\frontend
npm install
npm run build
```

This creates `frontend\dist`, which the backend serves automatically. Open `http://localhost:8000` after restarting the backend to see the login page.

Step 7: Install the Windows service

Open cmd as **Administrator**:

```
mkdir D:\IP_Inventory_Mgmt\logs
cd /d D:\IP_Inventory_Mgmt\tools
nssm install IPInventory "D:\IP_Inventory_Mgmt\venv\Scripts\python.exe" "-m uvicorn
app.main:app --host 0.0.0.0 --port 8000"
nssm set IPInventory AppDirectory D:\IP_Inventory_Mgmt\backend
nssm set IPInventory AppEnvironmentExtra
DATABASE_URL=mysql+pymysql://USER:ENCODED_PASSWORD@localhost:3306/ip_inventory
JWT_SECRET=YOUR_LONG_RANDOM_SECRET
nssm set IPInventory Start SERVICE_AUTO_START
nssm set IPInventory AppStdout D:\IP_Inventory_Mgmt\logs\service.log
nssm set IPInventory AppStderr D:\IP_Inventory_Mgmt\logs\service.log
nssm start IPInventory
nssm status IPInventory
```

The status must be `SERVICE_RUNNING`. To make the service wait for MySQL at boot, find the MySQL service name (`sc query type= service state= all | findstr /i mysql`) and run `nssm set IPInventory DependOnService MySQL80` using the real name.

Step 8: Open the firewall and share the address

```
netsh advfirewall firewall add rule name="IP Inventory" dir=in action=allow protocol=TCP
localport=8000
ipconfig
```

Colleagues open `http://SERVER-IP:8000`. Ask the network team to reserve the server's IP permanently.

Step 9: Create the first users

Log in as **admin**, open the **Users** tab, and add the Normal Users. Then create the first VLAN on the **VLANs** tab.

6. Moving existing data from the old PC

Instead of Step 3's `setup_database.sql`, restore a backup of the old database. This keeps all VLANs, IPs, users, passwords and the audit log.

On the old PC:

```
mysqldump -u root -p --single-transaction ip_inventory > ip_inventory_backup.sql
```

On the new PC (after MySQL is installed):

```
mysql -u root -p -e "CREATE DATABASE ip_inventory CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;"
mysql -u root -p ip_inventory < ip_inventory_backup.sql
```

Users keep their existing passwords. A new `JWT_SECRET` simply logs everyone out once.

7. Daily operation

Task	What to do
Check the service	<code>nssm status IPInventory</code>

Task	What to do
Restart after a backend change	Copy the file in, then <code>nssm restart IPInventory</code> (Administrator cmd)
Update the website	<code>npm run build</code> in frontend, then refresh the browser. No restart needed.
Read errors	<code>type D:\IP_Inventory_Mgmt\logs\service.log</code>
Change the database password or secret	Re-run the <code>nssm set IPInventory AppEnvironmentExtra ...</code> line, then <code>nssm restart IPInventory</code>
Reset a forgotten admin password	In a window with <code>DATABASE_URL</code> set: <code>python set_password.py admin NewPassword</code>

8. Backups (do this on day one)

The database is the only copy of the inventory.

1. Create `D:\IP_Inventory_Mgmt\backup.cnf` containing:

```
[client]
user=YOUR_DB_USER
password="YOUR_DB_PASSWORD"
```

2. Copy `backend\backup.bat` to `D:\IP_Inventory_Mgmt\` and check the `mysqldump.exe` path inside it (where `mysqldump` shows it).
3. Run it once and confirm a `.sql` file appears in `D:\IP_Inventory_Backups`.
4. Schedule it daily:

```
schtasks /create /tn "IP Inventory Backup" /tr "D:\IP_Inventory_Mgmt\backup.bat" /sc daily /st 22:00
```

5. Copy the backup folder to another computer or drive regularly, and test a restore at least once (`mysql -u root -p test_db < backup.sql`).

9. API reference

Interactive documentation is available at <http://SERVER-IP:8000/docs>. All endpoints except login and health need a login token.

Area	Endpoints	Who
Auth	POST <code>/api/auth/login</code> , GET <code>/api/auth/me</code> , POST <code>/api/auth/change-password</code>	Everyone
Users	GET/POST <code>/api/users</code> , PATCH <code>/api/users/{id}</code> , POST <code>/api/users/{id}/password</code>	Admin
VLANs	GET/POST <code>/api/vlans</code> , POST <code>/api/vlans/preview</code> , GET/PATCH <code>/api/vlans/{id}</code>	Admin
IPs	GET <code>/api/ips</code> , GET <code>/api/ips/vlans</code> , GET <code>/api/ips/available</code> , POST <code>/api/ips/{id}/assign</code> , <code>/release</code> , <code>/status</code> , PATCH <code>/api/ips/{id}</code>	Everyone (reserve/unreserve: Admin)
Audit	GET <code>/api/audit</code>	Admin
Reports	GET <code>/api/dashboard</code> , GET <code>/api/export/ips.csv</code>	Everyone
Health	GET <code>/api/health</code>	Public

10. Database tables

Table	Purpose
users	Accounts, bcrypt password hashes, role (admin / user), active flag
vlangs	VLAN ID, name, network, CIDR, range start/end (as integers), gateway, DNS, location, department, soft-delete flag
ip_addresses	One row per usable IP: status (available, assigned, reserved, inactive), department, room, equipment ID and name, CPU S/N, instrument S/N, user name, remarks, who assigned and when
audit_logs	Who, what, when, with old and new values (passwords are never logged)

IP addresses are stored as integers; convert with `INET_NTOA(ip_address)` in SQL. Useful query for who changed what:

```
SELECT a.created_at, u.username AS changed_by, a.action, a.entity_type, a.entity_id
FROM audit_logs a JOIN users u ON u.id = a.user_id
ORDER BY a.id DESC LIMIT 50;
```

11. Troubleshooting

Symptom	Cause and fix
'python' / 'npm' / 'nssm' is not recognized	Not installed, not on PATH, or you need a new cmd window. For nssm, cd /d into its folder first.
No module named 'app'	Run commands from the backend folder.
No module named 'fastapi' (or any package)	Activate the venv, then <code>pip install -r backend\requirements.txt</code> .
Access denied for user ...	Wrong password in DATABASE_URL, or @ not written as %40, or the set line was run in a different window.
Login page says "Login failed"	The backend returned an error. Read logs\service.log; usually a database connection problem.
"Incorrect username or password"	Backend and database work. Reset with <code>set_password.py</code> .
address already in use / port 8000	An old uvicorn window or service is still running. Stop it.
Page not reachable from other PCs	Firewall rule missing, wrong IP, or Wi-Fi client isolation. Test <code>ping SERVER-IP</code> and <code>http://SERVER-IP:8000/api/health</code> .
"Invalid network: ... has host bits set"	Network address must end at the start of the subnet (10.0.11.0/24).
Frontend shows the Vite starter page	Project files were not copied into frontend\src.
Service stops after reboot	Add the MySQL dependency (Step 7) and read <code>service.log</code> .
Empty file after paste/copy	Check sizes with <code>dir</code> ; a 0-byte .py file causes import errors.

12. Security recommendations

1. Use a dedicated MySQL user instead of root:

```
CREATE USER 'ipapp'@'localhost' IDENTIFIED BY 'a-long-random-password';
GRANT SELECT, INSERT, UPDATE, DELETE ON ip_inventory.* TO 'ipapp'@'localhost';
FLUSH PRIVILEGES;
```

Then update `DATABASE_URL` in the service and restart it. (Backups also work with this user using `--no-tablespaces`.)

2. Use a **long random** ``JWT_SECRET``, never a short or default one. Anyone who knows it can forge an admin login.
3. The service stores its settings in the Windows registry, readable only by administrators. Limit who has admin rights on this PC.
4. The system uses **plain HTTP**. That is acceptable on a trusted internal network, but passwords travel unencrypted. Add HTTPS through a reverse proxy (Caddy or IIS) if the network is shared or untrusted.
5. Set strong passwords for all accounts. Deactivate leavers on the **Users** tab.
6. Never share passwords or secrets in chat, email or screenshots.
7. Keep Windows, MySQL and Python updated, and keep the backups off the server.